PROGRAMACIÓN II · UNIDAD 1 · CLASE 11

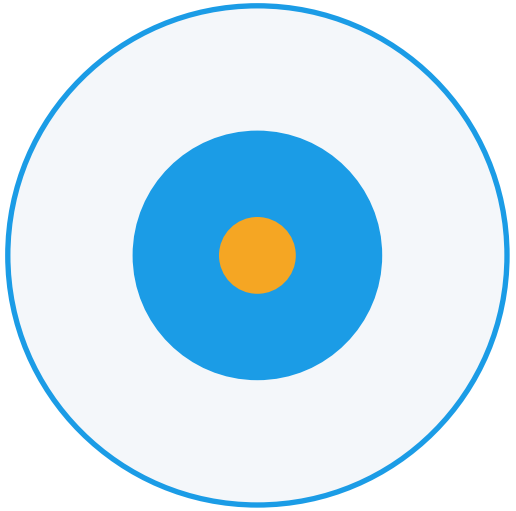
Enumerados y excepciones

Bloque 6 · Programación Orientada a Objetos en C#

> *Tipar valores fijos con enums y controlar el flujo del programa ante errores.*

Objetivo de la clase

Tipar valores fijos con **enums** y controlar el flujo del programa ante **errores**.



Qué veremos hoy

1. Enumerados (enum)
2. Tipos de fallas
3. Control de excepciones
4. Excepciones personalizadas

1 Enumerados (enum)

Un tipo con un conjunto de valores fijos y con nombre.

Enum.cs

```
enum DiaSemana { Lunes, Martes, Miercoles,
                 Jueves, Viernes, Sabado, Domingo }

DiaSemana hoy = DiaSemana.Sabado;

switch (hoy)
{
    case DiaSemana.Sabado:
    case DiaSemana.Domingo:
        Console.WriteLine("Fin de semana");
        break;
    default:
        Console.WriteLine("Día laboral");
        break;
}
```

- Define valores fijos y legibles, en vez de números sueltos.
- Se combina muy bien con switch para ramificar por cada valor.
- Varios case seguidos comparten una misma acción.

Apunte Unidad 1, pág. 45-46

2 Tipos de fallas

Compilación

Sintácticos y semánticos

Código mal escrito o sin sentido para el compilador. Los detecta en tiempo de compilación.

Ejecución

De lógica

El programa compila y corre, pero produce resultados incorrectos. Los más difíciles de detectar.

Ejecución

Eventos externos

Situaciones fuera del control del programa: archivo inexistente, división por cero, entrada inválida.

Los eventos externos son los que se manejan con excepciones: no se pueden evitar al escribir el código, pero sí controlar cuando ocurren.

Apunte Unidad 1, pág. 46-47

3 Control de excepciones: try / catch / finally

try-catch.cs

```
try
{
    int r = Dividir(a, b);   // puede fallar
}
catch (DivideByZeroException ex)
{
    Console.WriteLine("No se puede dividir por cero.");
}
finally
{
    Console.WriteLine("Siempre se ejecuta.");
}
```

- try: rodea el código que puede fallar.
- catch: captura y maneja una excepción de un tipo dado.
- finally: se ejecuta siempre, con o sin excepción (ideal para liberar recursos).
- throw: lanza una excepción, p. ej. throw new Exception("...").

Apunte Unidad 1, pág. 47-49

3 El orden de los catch

Se evalúan de arriba hacia abajo: del más específico al más general.

orden.cs

```
try { /* ... */ }
catch (DivideByZeroException ex) { }  // específico
catch (FormatException ex)       { }  // específico
catch (Exception ex)             { }  // general (último)
```

✓ **específico → general**

Exception (la base) va último.

✗ **general primero**

Un catch (Exception) al inicio tapa a los específicos: el compilador los marca como inalcanzables.

Apunte Unidad 1, pág. 47-49

4 Excepciones personalizadas

Errores propios del dominio: una clase que extiende Exception.

EmployeeException.cs

```
public class EmployeeException : Exception
{
    public EmployeeException(string message)
        : base(message) { }
}

// uso
if (name == null)
    throw new EmployeeException("Falta el nombre.");
```

- Extiende Exception (o una subclase).
- Reutiliza el mensaje del error con : base(message).
- Da errores con nombre propio del dominio: EmployeeException, EmployeeAttributeException.
- Se lanza con throw y se captura como cualquier excepción.

Apunte Unidad 1, pág. 50-53

4 Atrapar (A) vs. burbujear (B)

Cuando ocurre una excepción, hay dos opciones de diseño.

(A) Atrapar acá

El método resuelve el error en este nivel, con su propio try / catch.

(B) Dejar burbujear

No se captura acá: la excepción sube por la pila de llamadas hasta que alguien la maneje.

```
MetodoB (throw)  →  MetodoA  →  Main
```

Decisión de diseño: ¿este nivel sabe cómo manejar el error, o conviene que lo maneje un nivel superior?

Apunte Unidad 1, pág. 50-53

EN SÍNTESIS

- enum: un tipo con valores fijos y con nombre; se ramifica muy bien con switch.
- Tipos de fallas: sintácticos/semánticos, de lógica y eventos externos.
- Los eventos externos se manejan con try / catch / finally; throw lanza la excepción.
- El orden de los catch va del más específico al más general (Exception, última).
- Excepciones personalizadas extienden Exception; se puede atrapar (A) o dejar burbujear (B).

Próxima clase (práctica): enum + switch, try/catch/finally con throw y excepciones personalizadas (atrapar vs. burbujear).